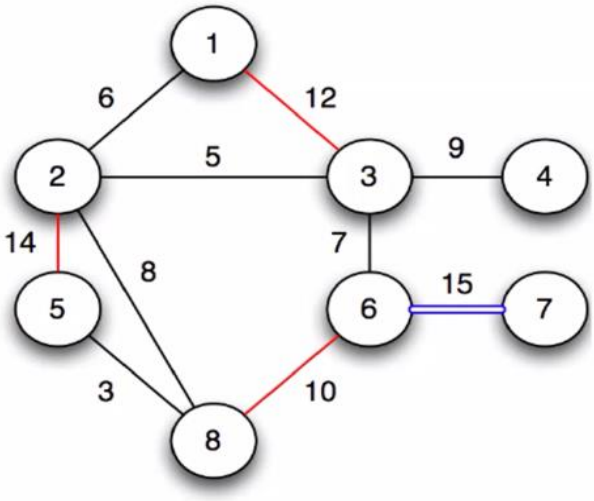
Kruskal's algorithm in action

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
yellow 15	(6,7)



$|V| = 8$

$|E| = 10$

$|T| = 7$

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \Longrightarrow \{1, 2, 3, 4, 5, 6, 7, 8\}$

Difference between Kruskal's and Prims Algorithm:

Kruskal's algorithm constructs multiple small trees and combines them, while Prim's algorithm grows a single tree starting from a chosen node.

BFS (Breadth First Search):
Kono graph k traverse korar por 2 ta jinis jante pari seta holo:
1. Source theke onno jekono targeted vertex e jete shortest path tar mane koita edge lage
2. Graph ta connected ki na. Jodi kono vertex still infinity thake tar mane sei vertex ta disconnected tai tak explore korte pare nai BFS technique r thik tokhonoi amra bujhe jabo j ei graph ta disconnected otherwise connected.

Sir k question korbo j j sob graph e direction dewa ache se sob graph e direction indication thaki kichu node e probesh kora jai na. Sekhetre ei graph ta ki disconnected bolte pari?

DFS(Depth First Search):
Kono graph k traverse korar por 2 ta jinis pai seta holo:
1. Discovery time
2. Finish time

Definition 1: A **graph** $G = (V, E)$ consists of V, a nonempty set of vertices (or nodes) and E, a set of edges. Each edge has either one or two vertices associated with it, called its endpoints. An edge is said to connect its endpoints.

🌳 Tree কী?

গ্রাফ থিওরিতে **Tree (Tree)** হলো এমন একটি graph—

- যেটা **connected** (সব node একে অপরের সাথে কোনো না কোনোভাবে যুক্ত)
- এবং এতে **কোনো cycle (loop)** নেই

📌 সহজভাবে:

☞ **Tree = Connected + No Cycle**

⋆ Tree-এর Main Characteristics:

1. No Cycle (কোনো loop নেই)

- কোনো node থেকে শুরু করে আবার সেই node-এ ফিরে আসা যায় না

2. Connected Graph

- সব node একে অপরের সাথে যুক্ত
- কোনো node আলাদা পড়ে থাকে না

3. Edges = Nodes − 1

যদি node সংখ্যা n হয়, তাহলে edge হবে:

n−1

4. Exactly One Path

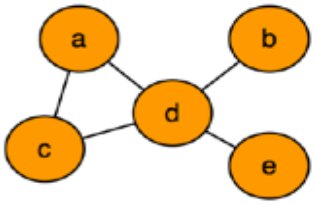
- যেকোনো দুইটি node এর মধ্যে **একটাই unique path** থাকে

Spanning trees

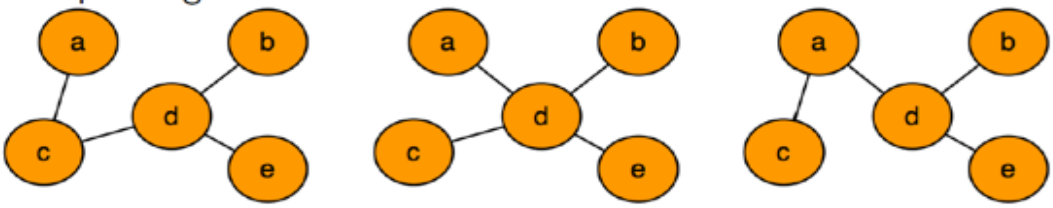
Definition

A subgraph T of a undirected graph $G = (V, E)$ is a spanning tree of G if it is a tree and contains every vertex of G.

Given the following graph:



The spanning trees are:



Minimum-cost Spanning Tree (MST)

Click to add text

Definition

The minimum-cost spanning tree of a graph A spanning tree T of a undirected graph $G = (V, E)$ is a minimum-cost spanning tree of G if the total weight $w(T) = \sum_{(u,v)\in T} w(u, v)$ is minimized.

How many edges will a tree have which contains total (2n2 + 3n - 4) nodes?
In graph theory, any **tree** with (N) nodes always has:
edges = N - 1

So, the number of edges will be:
(2n2+3n−4)−1=2n2+3n−5

So, the number of edges will be:
2n^2 + 3n - 5

✅ Final Answer:
2n^2 + 3n - 5

BFS will always visit every node of a tree. (True / False) – Justify your answer.
Answer: ✅ True

Justification:
Breadth-First Search (BFS) **is a traversal method that explores nodes level by level starting from a chosen root (or starting node).**

A **tree** is a connected graph with no cycles. Because it is **connected**, there is a path from the starting node to every other node.

- BFS begins at the root.
- It visits all neighboring nodes.
- Then it proceeds to their neighbors, and so on.

Since every node in a tree is reachable and there are no disconnected parts, BFS will **always visit every node exactly once**.

Key Point:

- BFS visits all nodes **if the graph is connected**.
- A tree is **always connected**, so BFS covers the entire tree.

1. Undirected Graph:

এখানে edge গুলোর কোনো direction (দিক) থাকে না।
অর্থাৎ, যদি A এবং B এর মধ্যে edge থাকে, তাহলে:

A↔BA \leftrightarrow BA↔B

মানে A থেকে B যাওয়া যায়, আবার B থেকে A-ও যাওয়া যায়।

2. Disconnected Graph

গ্রাফের সব node একে অপরের সাথে connected না থাকলে তাকে disconnected graph বলে। অর্থাৎ, এমন কিছু node বা group থাকে যাদের মধ্যে কোনো path নেই।

একসাথে (Undirected Disconnected Graph)

এটা এমন একটি graph:

- edge গুলোতে কোনো direction নেই (undirected)
- কিন্তু সব node একে অপরের সাথে connected না (disconnected)

উদাহরণ:

ধরো দুটি আলাদা অংশ আছে:

- Component 1: A — B — C
- Component 2: D — E

এখানে:

- A থেকে E যাওয়ার কোনো পথ নেই ✕
- তাই graph টি **disconnected**

গুরুত্বপূর্ণ কথা:

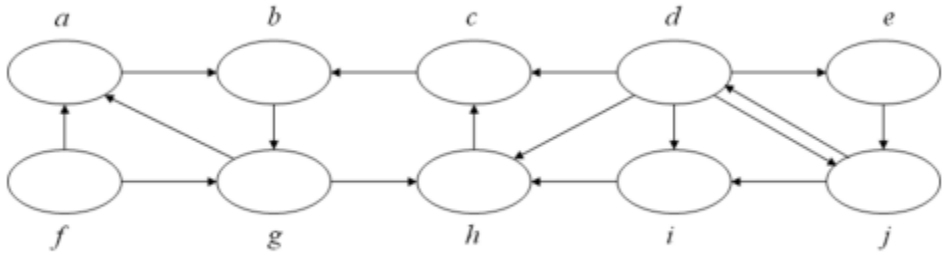
- এতে একাধিক **connected component** থাকে
- BFS বা DFS যদি একটি node থেকে শুরু করো, তাহলে শুধু ওই component টুকুই visit করবে, পুরো graph না

Important Concept:

- **Strongly Connected Graph**: সব node থেকে সব node এ যাওয়া যায়
- **Weakly Connected Graph OR Not strongly connected directed graph**: direction ignore করলে connected.

Weakly Connected Graph OR Not strongly connected directed graph

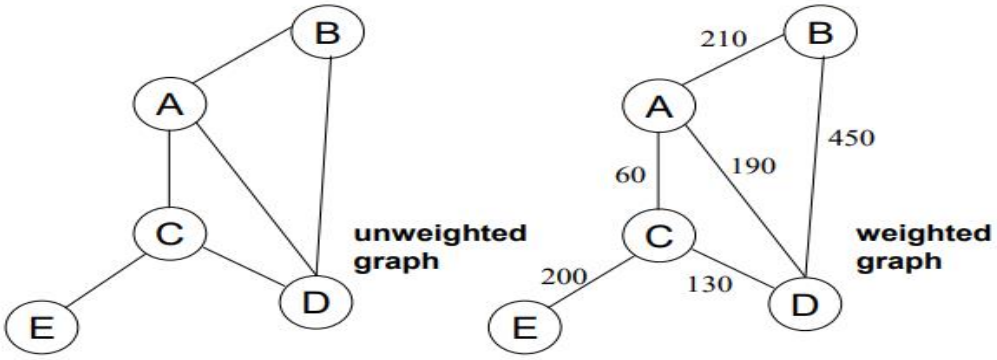
b) Consider the following graph G.



Apply BFS algorithm on it and compute shortest path distances of each vertex from ***d*** .

What is shortest path ?

- shortest length between two vertices for an unweighted graph:
- smallest **cost** between two vertices *for* a weighted graph:



Is the Minimum Spanning Tree always unique? – Justify your answer.

Answer: ✗ Not always unique

□ Explanation:

A **Minimum Spanning Tree (MST)** is a spanning tree with the smallest possible total edge weight.

🔑 When is MST unique?

☞ If **all edge weights are distinct (different)**
Then the MST is **always unique**

🔁 When is MST not unique?

☞ If **some edges have the same weight**, then:

- You can choose different edges (with equal weight)
- And still get the same minimum total cost

➞ So, multiple MSTs can exist

What happens if one extra edge is added to a Minimum Spanning Tree?

What happens if you add one extra edge to an MST?

A **Minimum Spanning Tree (MST)** is already:

- connected
- has no cycles
- has exactly n−1n - 1n−1 edges

✚ Add one extra edge:

☞ The moment you add **one more edge**, a **Cycle** is created.

□ Why?

- A tree has exactly one unique path between any two nodes
- When you add an extra edge between two nodes, you create a **second path**
- That automatically forms a **cycle**

Consider an undirected disconnected graph where every edge has same weight value of 2 . What information can we get if we apply BFS (Breadth-First Search) algorithm on this graph?

What happens if we apply BFS here?

You are given:

- An **Undirected Graph**
- It is **Disconnected Graph**
- All edge weights = 2 (same weight)

□ Key Idea:

Breadth-First Search (BFS) does **not use weights** — it only counts the number of edges (levels).

📌 So what information do we get?

1. Reachable Nodes (Connected Component)

- BFS will visit **only the nodes in the same component** as the starting node
- Other components will NOT be visited

2. Shortest Path (in terms of edges)

- BFS gives shortest path based on **number of edges**
- Since every edge has same weight (=2), this is also the **minimum cost path**

☞ Distance (cost) = (number of edges)×2

3. Level-wise Traversal

- BFS organizes nodes into levels:
  - Level 0 → start node
  - Level 1 → neighbors
  - Level 2 → neighbors of neighbors
- Helps understand structure of that component

! Important Limitation:

- BFS cannot reach nodes in other disconnected components
- So you don't get info about the **entire graph**, only one part

BFS vs DFS – Main Differences

🔥 BFS vs DFS – Main Differences

- **In BFS, a source vertex is specified, whereas in DFS, a source vertex is not necessarily specified.**
- **In BFS, all vertices may or may not be explored (for example, in a disconnected graph, some vertices may remain unvisited). However, in DFS, all vertices are eventually explored by applying DFS on each unvisited component.**
- **Since BFS starts from a given source vertex, the outcome is generally consistent. In contrast, DFS does not depend on a fixed starting point, so the outcome can vary depending on the traversal order.**

1. Traversal Style

- **BFS (Breadth First Search):**
  Explores nodes **level by level (layer-wise)**

- **DFS (Depth First Search):**
  Explores **as deep as possible**, then backtracks

2. Data Structure

- **BFS:** Uses **Queue (FIFO)**
- **DFS:** Uses **Stack / Recursion (LIFO)**

3. Visiting Order

- **BFS:** Visits **all adjacent (nearest) nodes first**
- **DFS:** Follows **one path deeply**, then explores others

🎯 Short Memory Trick

- **BFS = Level-wise + Queue**
- **DFS = Deep + Stack**

✅ One-line Exam Summary

☞ **BFS explores neighbors first, whereas DFS explores depth first before backtracking.**